Flight Explorer

Build a Roame-inspired flight list and flight map on both platforms: Swift/SwiftUI on iOS and Kotlin/Jetpack Compose on Android. Users should be able to discover flights, save options, and return to them offline.

The goal is to build two native experiences backed by a shared Kotlin Multiplatform data and persistence layer.

Getting Started

iOS development requires a Mac with Xcode. For Android, we recommend Android Studio. If you do not have a Mac, then using Compose Multiplatform for iOS UI is acceptable, but SwiftUI is strongly preferred.

We provide:

- Minimal SwiftUI and Compose apps displaying a shared KMP “Hello world!” string.
- Build configuration and setup instructions.
- JSON containing 1,000 fictional flights across 32 airports, with stable IDs, city names, airport coordinates, and departure dates from February 1 through April 1, 2027.

Each record already represents a dated flight. You do not need to generate data or implement schedule expansion.

You’ll build the flight UI, local API, and shared data layer. Choose your own libraries as needed. Follow the starter README and ensure both apps build before implementation.

What We Care About

Build a smooth, understandable experience: responsive scrolling and map gestures, consistent selection, and clear loading, empty, and error states.

System controls and simple layouts are welcome, but the app should feel cohesive and visually appealing. Pay attention to spacing, typography, hierarchy, and interaction feedback. Use Roame as a functional reference; an exact visual reproduction isn’t required.

Core Requirements

Local API

Implement `GET /flights`, returning HTTP 200 and the JSON envelope from `fixtures/flights.json`.

Use your preferred server language or framework. Document a single command to run the API locally.

Shared Data and Persistence

Implement networking, persistence, search, and filtering once in KMP, with both native clients consuming the shared logic.

Fetch flights from the API, persist the last successful results, and store saved flights locally. Each platform maintains its own local data; cross-device synchronization is not required.

Search and Filters

Provide separate origin and destination search fields that match airport codes or city names. Search should ignore capitalization and surrounding whitespace.

Support these filters:

- Departure date range, including both boundary dates.
- Cabin.
- Maximum points, including the selected limit.
- Loyalty program, allowing multiple selections.

Combine different filters using AND logic. A flight can match any selected loyalty program. Empty filters should include all values.

Show the result count and active filters, and provide a clear way to reset them. Start with the full fixture date range so results are visible immediately.

Flight List and Map

Both views should display the same filtered results and reflect the same saved state.

Group map markers by destination airport. Each marker should show the number of matching flights and the lowest points price. Tapping a marker should let the user browse those flights.

Support standard map gestures, including panning and zooming.

Flight Selection and Saved Flights

Selecting a flight should show its route connector and a card containing:

- Origin and destination.
- Departure date.
- Cabin.
- Points price.
- Loyalty program.
- Save or Remove action.

Preserve the selected flight when switching between list and map. Clear the selection if the flight is no longer included in the filtered results.

Provide a separate saved-flight view that remains accessible regardless of the current discovery filters.

Performance

The app should remain responsive with the full supplied dataset. Scrolling, searching, changing filters, and interacting with the map should feel smooth.

Network requests and database operations must not block the UI. Results should reflect the user's latest search and filter selections. There should be no crashes or memory leaks.

Offline and Error Handling

- Cached results and saved flights must survive closing and reopening the app.
- Users should be able to search cached results, view saved flights, and remove saved flights while offline.
- Failed refreshes should preserve existing data and show an offline or stale-data indication.
- If no cached data is available, show an understandable error with a retry action.
- Distinguish a successful empty response from a failed request.
- Saving the same flight again should update its details without creating duplicates.

- Saved flights should remain available even when absent from the latest API response.
- Failed save or remove operations must not silently appear successful.

Offline behavior applies to flight data. Downloading map tiles for offline use is not required.

Testing and Ownership

Add focused tests for shared filtering rules, date and points boundaries, map aggregation, persistence, and failure handling. Document the checks you actually performed on both platforms.

Include brief submission notes covering:

- Your approach and key architectural decisions.
- Assumptions and tradeoffs.
- Known limitations.

If any part of this project is ambiguous, document a reasonable assumption and proceed.

AI tools are welcome. Explain how you used them and how you checked their output. You should understand and be able to modify the code you submit.

Deliverables

Within two days, submit:

- A repository link or ZIP containing both apps, the shared KMP module, and your local API.
- Instructions for building, running, and testing the project.
- Your submission notes, including assumptions and tradeoffs, and how you used AI.
- A screen recording walkthrough demonstrating both apps, including search, filters, map selection, saving flights, and reopening the app with the API stopped.